Tucker Decomposition

Deep Understanding Guide for Questions 1–4

This document expands the section **Tucker Decomposition** from the uploaded exam guide. It assumes no prior tensor background, develops each question from zero, and follows the reference map listed in the guide.

Prepared as a single PDF study document

April 26, 2026

Contents

How to Use This Guide **2**

Zero-Knowledge Warm-Up: From Matrices to Tucker Tensors **2**

1 Existence of Tucker Decomposition, Minimal Decomposition, and Tucker Ranks **4**

2 HOSVD, Sequentially Truncated HOSVD, and Approximation Error Estimates **7**

3 Arithmetic in Tucker Format and Orthogonal Tucker Decomposition **10**

4 Orthogonalization and Quasi-Optimal Recompression in Tucker Format **13**

Cross-Question Concept Map **16**

Compact Exam Checklist **16**

Reference List Used from the Uploaded Guide **16**

How to Use This Guide

The Tucker section has four questions. They form a coherent progression:

1. What the Tucker decomposition is, why it always exists, and what the Tucker ranks mean.
2. How HOSVD and sequentially truncated HOSVD construct approximations and control error.
3. How arithmetic works in Tucker format and why orthogonal Tucker form is special.
4. How orthogonalization and recompression reduce ranks while preserving accuracy.

Main mental model. CP represents a tensor as a sum of rank-one outer products. Tucker instead represents a tensor as a small central core connected to one basis matrix per mode. The core says how the mode-wise basis vectors interact.

The attached guide points especially to Kolda–Bader for notation and the Tucker model, De Lathauwer–De Moor–Vandewalle for HOSVD and multilinear ranks, Vannieuwenhoven–Vandebril–Meerbergen for sequential truncation and error control, Tucker's original three-mode factor model, Golub–Van Loan for QR/SVD machinery, and Trefethen–Bau for basic orthogonality and matrix norms.

Zero-Knowledge Warm-Up: From Matrices to Tucker Tensors

A vector has one index, a matrix has two indices, and a tensor has several indices:

$$\mathbf{x} = (x_i), \qquad \mathbf{A} = (a_{ij}), \qquad \mathcal{X} = (x_{i_1 i_2 \cdots i_d}).$$

The number of indices is the **order** or **number of modes**. For example, a color video can be viewed as a tensor with modes such as height, width, color channel, and time.

For a matrix, low rank means that all columns lie in a low-dimensional column subspace and all rows lie in a low-dimensional row subspace. The singular value decomposition expresses this idea as

$$\mathbf{A} \approx \mathbf{U}\mathbf{S}\mathbf{V}^{\mathsf{T}}.$$

The columns of $\mathbf{U}$ are a basis for the important column directions, the columns of $\mathbf{V}$ are a basis for the important row directions, and the small diagonal matrix $\mathbf{S}$ tells how those directions interact.

Tucker decomposition is the tensor analogue of this idea. Instead of one row basis and one column basis, it uses one basis matrix for every mode:

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \cdots \times_d \mathbf{U}^{(d)}.$$

The tensor $\mathcal{G}$ is the **core**. The matrices $\mathbf{U}^{(n)}$ contain basis vectors for mode n. The symbol $\times_n$ means multiplication along mode n.

For a three-way tensor, the entries can be written as

$$x_{ijk} \approx \sum_{\alpha=1}^{r_1} \sum_{\beta=1}^{r_2} \sum_{\gamma=1}^{r_3} g_{\alpha\beta\gamma}\, u^{(1)}_{i\alpha}\, u^{(2)}_{j\beta}\, u^{(3)}_{k\gamma}.$$

This formula says: choose a few basis vectors in each mode, then combine them through the core coefficients.

Core intuition. A CP model has a diagonal-like interaction structure: component 1 interacts with component 1 in every mode, component 2 with component 2, and so on. Tucker is more flexible: its core allows any basis direction in one mode to interact with any basis direction in the other modes.

Important notation

Symbol	Meaning
$\mathcal{X}$	Original tensor.
$\mathcal{G}$	Tucker core, usually much smaller than $\mathcal{X}$.
$\mathbf{U}^{(n)}$	Factor matrix for mode n; its columns are mode-n basis vectors.
$\times_n$	Mode-n multiplication: multiply every mode-n fiber by a matrix.
$\mathbf{X}_{(n)}$	Mode-n unfolding: reshape the tensor into a matrix with mode n as rows.
$(r_1, \ldots, r_d)$	Tucker rank or multilinear rank.
$\|\mathcal{X}\|_{\mathrm{F}}$	Frobenius norm: square root of the sum of squared entries.

1 Existence of Tucker Decomposition, Minimal Decomposition, and Tucker Ranks

Question in plain language

Can every tensor be represented in Tucker form? What is the smallest meaningful Tucker representation, and how are Tucker ranks defined?

Start from zero

The Tucker decomposition generalizes a matrix factorization. For a matrix, one may write

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^{\mathsf{T}},$$

where $\mathbf{U}$ and $\mathbf{V}$ give bases and $\mathbf{S}$ stores interactions between those bases. Tucker decomposition does the same for a tensor:

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \cdots \times_d \mathbf{U}^{(d)}.$$

The core $\mathcal{G}$ is not usually diagonal. It is a multidimensional table of interaction coefficients.

For a three-way tensor, Tucker writes

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}.$$

Entrywise this means

$$x_{ijk} = \sum_{\alpha=1}^{r_1} \sum_{\beta=1}^{r_2} \sum_{\gamma=1}^{r_3} g_{\alpha\beta\gamma} a_{i\alpha} b_{j\beta} c_{k\gamma}.$$

The indices α, β, γ run over the reduced dimensions. If r_1, r_2, and r_3 are small, the representation is compact.

Why an exact Tucker decomposition always exists

Every finite tensor has an exact Tucker decomposition. The simplest proof is almost trivial: choose identity matrices as all factor matrices and let the core equal the original tensor:

$$\mathcal{X} = \mathcal{X} \times_1 \mathbf{I} \times_2 \mathbf{I} \cdots \times_d \mathbf{I}.$$

That representation is exact but not compressed. The real goal is to find small mode bases that still represent the tensor exactly, or approximately.

A more useful exact construction uses the column spaces of unfolding matrices. The mode-n unfolding $\mathbf{X}_{(n)}$ is a matrix made by putting index i_n in the rows and all other indices in the columns. If

$$r_n = \operatorname{rank}\big(\mathbf{X}_{(n)}\big),$$

then we can choose $\mathbf{U}^{(n)}$ to be any basis for the column space of $\mathbf{X}_{(n)}$. With such bases, the tensor lies inside the product of the selected mode subspaces, so an exact core exists.

Existence intuition. A tensor has one family of fibers in each mode. If you choose bases that span all mode-n fibers for every mode n, then the tensor can be reconstructed by combining those bases with a core.

Tucker rank, also called multilinear rank

The Tucker rank is the vector of ranks of the mode unfoldings:

$$\operatorname{rank}_{\text{Tucker}}(\mathcal{X}) = \big(\operatorname{rank}(\mathbf{X}_{(1)}), \operatorname{rank}(\mathbf{X}_{(2)}), \ldots, \operatorname{rank}(\mathbf{X}_{(d)})\big).$$

Unlike CP rank, Tucker rank is not a single number. It is a tuple because each mode can have a different intrinsic dimension.

For a three-way tensor, Tucker rank (r_1, r_2, r_3) means:

- all mode-1 fibers live in an r_1-dimensional subspace;
- all mode-2 fibers live in an r_2-dimensional subspace;
- all mode-3 fibers live in an r_3-dimensional subspace.

Minimal Tucker decomposition

A Tucker decomposition is minimal when the factor matrices have exactly these unfolding ranks:

$$\mathbf{U}^{(n)} \in \mathbb{R}^{I_n \times r_n}, \qquad r_n = \operatorname{rank}(\mathbf{X}_{(n)}).$$

Equivalently, no factor matrix can be made narrower without losing exactness. The minimal core has size

$$r_1 \times r_2 \times \cdots \times r_d.$$

This minimal size is well defined, although the factors themselves are not unique. Any invertible change of basis in a mode can be compensated inside the core.

Gauge freedom: why Tucker factors are not unique

Suppose $\mathbf{M}^{(n)}$ is an invertible $r_n \times r_n$ matrix. Then we can replace

$$\mathbf{U}^{(n)} \quad \text{by} \quad \mathbf{U}^{(n)}\mathbf{M}^{(n)}$$

and multiply the core along the same mode by $(\mathbf{M}^{(n)})^{-1}$. The full tensor does not change. Therefore Tucker decomposition is best understood as identifying mode subspaces and a coordinate-dependent core, not as a unique list of factor matrices.

Common confusion. Minimal Tucker decomposition is not the same as CP decomposition. CP rank asks for the minimum number of rank-one terms. Tucker rank asks for the dimensions of mode-wise subspaces. These are different notions of simplicity.

Connection with matrices

For a matrix, Tucker decomposition reduces to a familiar low-rank factorization:

$$\mathbf{A} = \mathbf{U}\mathbf{G}\mathbf{V}^{\mathsf{T}}.$$

Here $\mathbf{G}$ is a small matrix. If $\mathbf{U}$ and $\mathbf{V}$ are orthonormal bases for the column and row spaces, $\mathbf{G} = \mathbf{U}^{\mathsf{T}}\mathbf{A}\mathbf{V}$. In tensors, the same projection idea gives

$$\mathcal{G} = \mathcal{X} \times_1 (\mathbf{U}^{(1)})^{\mathsf{T}} \times_2 (\mathbf{U}^{(2)})^{\mathsf{T}} \cdots \times_d (\mathbf{U}^{(d)})^{\mathsf{T}},$$

when the factors are orthonormal.

What to know for the exam

You should be able to:

- write the Tucker form with a core and factor matrices;
- explain mode-n multiplication in words and with the three-way entry formula;
- define Tucker rank as the vector of ranks of mode unfoldings;
- explain why an exact Tucker decomposition always exists;
- define minimal Tucker decomposition and distinguish it from CP rank;
- mention non-uniqueness caused by changes of basis.

Reference map from the attached guide

- **KB09, pp. 459–464**: mode products, matricization, and tensor notation.
- **KB09, pp. 473–478**: Tucker decomposition, core tensor, and multilinear rank.
- **T66, pp. 279–288**: original three-mode factor-analysis viewpoint.
- **DL00, pp. 1256–1264**: mode vectors, ranks, and HOSVD preliminaries.

One-sentence answer. Tucker decomposition always exists; its minimal exact version uses mode subspaces whose dimensions are the ranks of the mode unfolding matrices, and these dimensions form the Tucker or multilinear rank.

2 HOSVD, Sequentially Truncated HOSVD, and Approximation Error Estimates

Question in plain language

How do we compute a useful Tucker approximation, and how can we estimate the approximation error?

Start from zero: SVD as the matrix prototype

The matrix SVD gives the best rank-r approximation:

$$\mathbf{A} \approx \mathbf{U}_r \mathbf{S}_r \mathbf{V}_r^{\mathsf{T}}.$$

It chooses the r most important left singular vectors and the r most important right singular vectors. The approximation error is exactly controlled by the discarded singular values.

HOSVD means **higher-order singular value decomposition**. It copies the matrix idea mode by mode. Instead of applying SVD to the tensor directly, it applies matrix SVDs to the unfolding matrices $\mathbf{X}_{(n)}$.

HOSVD algorithm

Given a tensor $\mathcal{X}$ and desired Tucker ranks $(r_1, \ldots, r_d)$:

1. For each mode n, form the unfolding matrix $\mathbf{X}_{(n)}$.
2. Compute the leading r_n left singular vectors of $\mathbf{X}_{(n)}$.
3. Put these vectors into $\mathbf{U}^{(n)}$.
4. Project the tensor onto the chosen subspaces:

$$\mathcal{G} = \mathcal{X} \times_1 (\mathbf{U}^{(1)})^{\mathsf{T}} \times_2 (\mathbf{U}^{(2)})^{\mathsf{T}} \cdots \times_d (\mathbf{U}^{(d)})^{\mathsf{T}}.$$

5. Reconstruct the approximation:

$$\widehat{\mathcal{X}} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \times_2 \mathbf{U}^{(2)} \cdots \times_d \mathbf{U}^{(d)}.$$

HOSVD intuition. Look at the tensor from each mode separately. In each view, keep the strongest singular directions. Then build the tensor approximation inside the product of those chosen subspaces.

Why HOSVD is not generally optimal

For matrices, truncated SVD is optimal. For tensors, choosing the best subspaces jointly is harder because the modes interact. HOSVD chooses each mode subspace independently from its unfolding. This is sensible and stable, but it does not always solve the fully optimal Tucker approximation problem:

$$\min_{\operatorname{rank}_{\text{Tucker}}(\mathcal{Y}) \le (r_1, \ldots, r_d)} \|\mathcal{X} - \mathcal{Y}\|_{\mathrm{F}}.$$

The true best Tucker approximation may require mode subspaces that are not simply the leading singular vectors of the original unfoldings. However, HOSVD is **quasi-optimal**: its error is within a known factor of the best possible error.

Error from discarded singular values

Let ε_n be the truncation error from mode n, defined by the singular values of $\mathbf{X}_{(n)}$ that were discarded:

$$\varepsilon_n^2 = \sum_{j>r_n} (\sigma_j^{(n)})^2.$$

Then the HOSVD approximation satisfies the important estimate

$$\|\mathcal{X} - \widehat{\mathcal{X}}\|_{\mathrm{F}} \leq \sqrt{\varepsilon_1^2 + \varepsilon_2^2 + \cdots + \varepsilon_d^2}.$$

This bound says that each mode truncation contributes to the total error, and the total is controlled by the root-sum-square of the mode-wise discarded tails.

If $\mathcal{X}_{\text{best}}$ denotes the best Tucker approximation with ranks $(r_1, \ldots, r_d)$, then a common quasi-optimal form is

$$\|\mathcal{X} - \widehat{\mathcal{X}}\|_{\mathrm{F}} \leq \sqrt{d}\, \|\mathcal{X} - \mathcal{X}_{\text{best}}\|_{\mathrm{F}}.$$

The exact constant depends on the formulation and truncation setting, but the key exam point is that HOSVD is controlled and quasi-optimal, not necessarily optimal.

Sequentially truncated HOSVD

Sequentially truncated HOSVD, often written st-HOSVD, performs mode truncations one after another. After truncating one mode, it updates the tensor and uses the smaller intermediate tensor for later SVDs.

A typical st-HOSVD procedure is:

1. Set $\mathcal{Y} = \mathcal{X}$.
2. For $n = 1, 2, \ldots, d$:
   (a) unfold the current tensor $\mathcal{Y}$ in mode n;
   (b) compute leading left singular vectors;
   (c) multiply $\mathcal{Y}$ by the transpose of those vectors along mode n.
3. The final $\mathcal{Y}$ is the core, and the collected singular vector matrices are the factors.

Why sequential truncation can be cheaper. After each projection, the tensor becomes smaller in one mode. Later SVDs are therefore applied to smaller matrices than in full HOSVD.

Choosing ranks from a tolerance

Suppose the target error is ε. A common strategy is to choose per-mode tolerances δ_n such that

$$\delta_1^2 + \delta_2^2 + \cdots + \delta_d^2 \leq \varepsilon^2.$$

For simplicity, if all modes are treated equally, one can use

$$\delta_n = \frac{\varepsilon}{\sqrt{d}}.$$

Then choose each r_n so that the discarded singular value tail in mode n is at most δ_n.

All-orthogonality and the HOSVD core

When the factors $\mathbf{U}^{(n)}$ are orthonormal, HOSVD has properties similar to matrix SVD. The core has orthogonality-like structure across slices, and its norm is preserved by the orthonormal factor matrices:

$$\|\mathcal{G}\|_{\mathrm{F}} = \|\widehat{\mathcal{X}}\|_{\mathrm{F}}.$$

This makes error estimates and rank truncation easier to reason about.

HOSVD versus st-HOSVD

Feature	**HOSVD**	**st-HOSVD**
Main idea	Compute SVDs of original unfoldings.	Truncate modes sequentially and work with smaller intermediates.
Cost	Can be expensive because all original unfoldings may be large.	Often cheaper because later unfoldings are smaller.
Error control	Root-sum-square of discarded mode tails.	Similar controlled truncation, but tails are computed from updated tensors.
Output	Orthogonal Tucker approximation.	Orthogonal Tucker approximation, usually more efficient.
Best use	Conceptual clarity and direct analysis.	Practical large-scale compression.

What to know for the exam

You should be able to:

- describe HOSVD step by step;
- state that HOSVD uses leading left singular vectors of mode unfoldings;
- explain why it is not necessarily the optimal Tucker approximation;
- state the root-sum-square error estimate from discarded singular value tails;
- explain sequential truncation and why it reduces computational cost;
- explain rank selection from a global tolerance.

Reference map from the attached guide

- **DL00, pp. 1256–1268**: HOSVD theorem, computation by unfoldings, and properties.
- **KB09, pp. 476–478**: compact HOSVD algorithm and quasi-best approximation discussion.
- **VVM12, pp. A1027–A1038**: error expressions and motivation for sequential truncation.
- **VVM12, pp. A1038–A1052**: st-HOSVD algorithm, rank truncation, and experiments.

One-sentence answer. HOSVD constructs Tucker factors from the dominant left singular vectors of mode unfoldings; st-HOSVD performs these truncations sequentially on smaller tensors, and both use discarded singular value tails to control the Frobenius error.

3 Arithmetic in Tucker Format and Orthogonal Tucker Decomposition

Question in plain language

Once tensors are stored in Tucker form, how do we add them, multiply them by matrices, take scalar products, and compute norms efficiently? Why is orthogonality important?

Start from zero: why arithmetic matters

A low-rank format is useful only if we can operate on it without expanding the full tensor. If $\mathcal{X}$ has dimensions $I_1 \times \cdots \times I_d$, the full number of entries is

$$I_1 I_2 \cdots I_d.$$

This can be enormous. Tucker storage is much smaller when ranks are moderate:

$$\text{storage} \approx r_1 r_2 \cdots r_d + \sum_{n=1}^{d} I_n r_n.$$

The first term stores the core. The second term stores the factor matrices.

Applying a matrix along one mode

Suppose

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \times_2 \cdots \times_d \mathbf{U}^{(d)}.$$

If we multiply the full tensor in mode k by a matrix $\mathbf{M}$, we get

$$\mathcal{Y} = \mathcal{X} \times_k \mathbf{M}.$$

In Tucker form, this simply changes the mode-k factor:

$$\mathcal{Y} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \cdots \times_k \left(\mathbf{M}\mathbf{U}^{(k)}\right) \cdots \times_d \mathbf{U}^{(d)}.$$

So mode-wise linear transformations are cheap and natural.

Addition of two Tucker tensors

Suppose

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{U}^{(1)} \cdots \times_d \mathbf{U}^{(d)}$$

and

$$\mathcal{Y} = \mathcal{H} \times_1 \mathbf{V}^{(1)} \cdots \times_d \mathbf{V}^{(d)}.$$

Their sum can be represented exactly by concatenating factor matrices:

$$\mathbf{W}^{(n)} = \begin{bmatrix} \mathbf{U}^{(n)} & \mathbf{V}^{(n)} \end{bmatrix}.$$

The new core is block structured: $\mathcal{G}$ occupies one diagonal block of the multidimensional core, $\mathcal{H}$ occupies the other, and mixed blocks are zero. For a matrix, this is analogous to writing

$$\mathbf{U}\mathbf{G}\mathbf{V}^\top + \mathbf{P}\mathbf{H}\mathbf{Q}^\top = \begin{bmatrix} \mathbf{U} & \mathbf{P} \end{bmatrix} \begin{bmatrix} \mathbf{G} & 0 \\ 0 & \mathbf{H} \end{bmatrix} \begin{bmatrix} \mathbf{V} & \mathbf{Q} \end{bmatrix}^\top.$$

Rank growth after addition. Addition is easy, but it increases the Tucker ranks from $(r_1, \ldots, r_d)$ and $(s_1, \ldots, s_d)$ to at most $(r_1 + s_1, \ldots, r_d + s_d)$. Recompression is usually needed afterward.

Scalar products

The scalar product of two full tensors is

$$\langle \mathcal{X}, \mathcal{Y} \rangle = \sum_{i_1,\dots,i_d} x_{i_1\cdots i_d} y_{i_1\cdots i_d}.$$

In Tucker format, this can be computed by contracting small objects. The factor matrices first produce cross-Gram matrices

$$\mathbf{M}^{(n)} = (\mathbf{U}^{(n)})^{\mathsf{T}} \mathbf{V}^{(n)}.$$

Then one contracts the two cores through these small matrices. Conceptually:

$$\langle \mathcal{X}, \mathcal{Y} \rangle = \langle \mathcal{G}, \mathcal{H} \times_1 \mathbf{M}^{(1)} \times_2 \cdots \times_d \mathbf{M}^{(d)} \rangle.$$

This avoids forming $\mathcal{X}$ and $\mathcal{Y}$ explicitly.

Norms and orthogonal Tucker decomposition

A Tucker decomposition is called orthogonal if every factor matrix has orthonormal columns:

$$(\mathbf{U}^{(n)})^{\mathsf{T}} \mathbf{U}^{(n)} = \mathbf{I}.$$

When this holds, the Frobenius norm of the full tensor equals the Frobenius norm of the core:

$$\|\mathcal{X}\|_{\mathrm{F}} = \|\mathcal{G}\|_{\mathrm{F}}.$$

This is extremely useful because the core is small.

Why orthogonality helps. Orthogonal factor matrices preserve lengths and angles. Therefore they do not distort the Frobenius norm when the core is expanded into the full tensor.

Projection interpretation

If $\mathbf{U}^{(n)}$ has orthonormal columns, then

$$\mathbf{P}^{(n)} = \mathbf{U}^{(n)} (\mathbf{U}^{(n)})^{\mathsf{T}}$$

is the orthogonal projector onto the chosen mode-n subspace. The Tucker approximation

$$\widehat{\mathcal{X}} = enX \times_1 \mathbf{P}^{(1)} \times_2 \mathbf{P}^{(2)} \cdots \times_d \mathbf{P}^{(d)}$$

is the tensor after projecting each mode into the selected subspace.

This projection view explains both HOSVD and the norm/error formulas. The factors specify subspaces; the core is the coordinate representation of the projected tensor inside those subspaces.

Hadamard and multilinear operations

Elementwise products, contractions, and nonlinear elementwise functions are more complicated than addition or mode multiplication. The exact Tucker ranks can grow quickly because new interactions appear in the core and across factor spaces. In practice, one often computes an enlarged representation and then recompresses it.

Comparison with CP arithmetic

CP addition is just concatenation of components. Tucker addition also concatenates bases, but its core becomes block structured. Tucker can be more stable for arithmetic because ranks are tied to closed subspace dimensions, while CP rank can behave pathologically. However, Tucker cores scale as $r_1 r_2 \cdots r_d$, so Tucker can become expensive when the number of modes is large.

What to know for the exam

You should be able to:

- give the Tucker storage estimate;
- describe mode-wise multiplication by updating one factor matrix;
- describe addition through concatenated factors and block cores;
- explain rank growth after arithmetic;
- compute or describe scalar products through small Gram matrices and core contractions;
- explain why orthogonal factors make norm computation simple.

Reference map from the attached guide

- **KB09, pp. 473–480**: Tucker notation, core, factor matrices, and matricized forms.
- **DL00, pp. 1258–1268**: all-orthogonality, norm, and SVD-like properties of HOSVD.
- **VVM12, pp. A1027–A1034**: orthogonal Tucker approximation and error formulas.
- **TB97, pp. 1–45**: orthogonality, norms, and SVD background.

One-sentence answer. Tucker arithmetic is performed on cores and factor matrices: mode products update factors, sums concatenate factor spaces and form block cores, scalar products use small Gram contractions, and orthogonal factors make the full tensor norm equal to the core norm.

4 Orthogonalization and Quasi-Optimal Recompression in Tucker Format

Question in plain language

After arithmetic increases Tucker ranks or produces non-orthogonal factors, how can we convert the representation into a stable orthogonal form and reduce the ranks while controlling the error?

Start from zero: why recompression is needed

Many operations increase ranks. For example, adding two Tucker tensors can double the ranks. Repeating operations without compression eventually makes the representation too large. Recompression means replacing a large Tucker representation by a smaller one that is exactly equal or approximately equal within a prescribed tolerance.

There are two related tasks:

- **Orthogonalization**: make the factor matrices orthonormal while preserving the tensor exactly.
- **Truncation**: reduce the Tucker ranks, usually by SVD, while controlling the error.

Orthogonalization by QR

Suppose the factor matrix in mode n is not orthonormal. Compute a QR factorization:

$$\mathbf{U}^{(n)} = \mathbf{Q}^{(n)}\mathbf{R}^{(n)},$$

where $\mathbf{Q}^{(n)}$ has orthonormal columns. Then

$$\mathcal{G} \times_n \mathbf{U}^{(n)} = \mathcal{G} \times_n (\mathbf{Q}^{(n)}\mathbf{R}^{(n)}) = (\mathcal{G} \times_n \mathbf{R}^{(n)}) \times_n \mathbf{Q}^{(n)}.$$

So we replace the factor by $\mathbf{Q}^{(n)}$ and absorb $\mathbf{R}^{(n)}$ into the core. Repeating this for every mode gives an orthogonal Tucker representation of the same tensor.

Orthogonalization intuition. QR separates a factor into a clean orthonormal basis and a small coordinate transformation. The basis stays as the factor; the coordinate transformation is pushed into the core.

Why orthogonalization should come before truncation

When factors are orthonormal, the Frobenius norm of the tensor equals the Frobenius norm of the core. Therefore the approximation error caused by truncating the core or its unfoldings can be measured in the small representation. Without orthogonality, the factor matrices can distort errors, and small core error may become large full-tensor error.

Recompression by HOSVD of the small core

After orthogonalization, the tensor has the form

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{Q}^{(1)} \times_2 \mathbf{Q}^{(2)} \cdots \times_d \mathbf{Q}^{(d)}.$$

If the ranks are too large, apply HOSVD or st-HOSVD to the core $\mathcal{G}$, not to the full tensor. Suppose we approximate

$$\mathcal{G} \approx \mathcal{S} \times_1 \mathbf{W}^{(1)} \times_2 \mathbf{W}^{(2)} \cdots \times_d \mathbf{W}^{(d)}.$$

Then substitute into the full tensor:

$$\mathcal{X} \approx \mathcal{S} \times_1 (\mathbf{Q}^{(1)}\mathbf{W}^{(1)}) \times_2 (\mathbf{Q}^{(2)}\mathbf{W}^{(2)}) \cdots \times_d (\mathbf{Q}^{(d)}\mathbf{W}^{(d)}).$$

The new factors are narrower, and the new core is smaller.

Quasi-optimality

A recompression is quasi-optimal if its error is within a moderate known factor of the best possible Tucker approximation at the chosen ranks. For HOSVD-type truncation, the standard message is:

$$\|\mathcal{X} - \widehat{\mathcal{X}}\|_{\mathrm{F}} \leq \sqrt{d}\,\|\mathcal{X} - \mathcal{X}_{\mathrm{best}}\|_{\mathrm{F}}.$$

This is not as strong as the matrix truncated SVD, which is exactly optimal, but it is strong enough for reliable compression algorithms.

Meaning of quasi-optimal. The algorithm may not find the best possible tensor with the target Tucker ranks, but its error is provably not much worse than the best possible error.

Distributing a tolerance across modes

If the desired recompression tolerance is ε, choose mode-wise truncation thresholds δ_n satisfying

$$\sum_{n=1}^{d} \delta_n^2 \leq \varepsilon^2.$$

The simplest equal distribution is

$$\delta_n = \frac{\varepsilon}{\sqrt{d}}.$$

In practice, unequal distributions may be better if some modes have much faster singular value decay than others.

Algorithm: orthogonal Tucker recompression

A practical recompression algorithm is:

1. Start with a Tucker tensor $\mathcal{G} \times_1 \mathbf{U}^{(1)} \cdots \times_d \mathbf{U}^{(d)}$.
2. For each mode n, compute $\mathbf{U}^{(n)} = \mathbf{Q}^{(n)}\mathbf{R}^{(n)}$.
3. Replace $\mathbf{U}^{(n)}$ by $\mathbf{Q}^{(n)}$ and absorb $\mathbf{R}^{(n)}$ into the core along mode n.
4. Apply HOSVD or st-HOSVD to the now-small orthogonal core.
5. Multiply the old orthogonal factors by the newly obtained core factors.
6. Return the smaller Tucker tensor.

Why this avoids forming the full tensor

All expensive operations happen on factor matrices and the core. If the original dimensions I_n are large but ranks r_n are moderate, QR factorizations cost roughly like operations on $I_n \times r_n$ matrices, while truncation SVDs happen on unfoldings of the small core. The full tensor with $I_1 I_2 \cdots I_d$ entries is never formed.

Numerical stability

Orthogonalization also improves numerical stability. Non-orthogonal factor matrices can become ill-conditioned, meaning that small changes in the core may produce large changes in the full tensor. Orthonormal bases avoid this amplification. This is one reason orthogonal Tucker format is the standard setting for truncation and error estimation.

What to know for the exam

You should be able to:

- explain why rank growth forces recompression;
- describe QR orthogonalization of Tucker factors;
- show how the **R** factors are absorbed into the core;
- explain why orthogonality makes norms and errors easy to control;
- describe core recompression using HOSVD or st-HOSVD;
- state the meaning of quasi-optimal recompression;
- explain tolerance distribution across modes.

Reference map from the attached guide

- **DL00, pp. 1260–1268**: orthogonal HOSVD structure and norm properties.
- **KB09, pp. 476–480**: truncated Tucker/HOSVD computation.
- **VVM12, pp. A1027–A1052**: T-HOSVD, st-HOSVD, and error-controlled truncation.
- **GVL13, pp. 248–288**: QR/SVD machinery behind orthogonalization.

One-sentence answer. Tucker recompression first orthogonalizes factors by QR and pushes the triangular parts into the core, then truncates the small orthogonal core by HOSVD or st-HOSVD, giving a smaller representation with controlled quasi-optimal error.

Cross-Question Concept Map

Concept	Meaning	Where it appears
Tucker core	Small tensor of interaction coefficients between mode bases.	Questions 1–4.
Mode factors	Basis matrices for each tensor mode.	Questions 1–4.
Tucker rank	Vector of unfolding ranks.	Question 1.
Mode unfolding	Matrix view of the tensor used to define ranks and compute SVDs.	Questions 1–2.
HOSVD	Tucker approximation from SVDs of mode unfoldings.	Question 2.
st-HOSVD	Sequential truncation variant that works with progressively smaller tensors.	Questions 2 and 4.
Orthogonal Tucker form	Tucker form with orthonormal factor matrices.	Questions 2–4.
Recompression	Reducing ranks after arithmetic while preserving accuracy.	Questions 3–4.
Quasi-optimality	Error within a known factor of the best possible fixed-rank Tucker approximation.	Questions 2 and 4.

Compact Exam Checklist

Before the exam, make sure you can answer the following without notes:

1. What is the Tucker decomposition, and what do the core and factors represent?
2. Why does an exact Tucker decomposition always exist?
3. What is the Tucker rank, and how is it connected to the ranks of mode unfoldings?
4. How does HOSVD choose the factor matrices?
5. Why is HOSVD quasi-optimal rather than generally optimal?
6. How do discarded singular values control the Frobenius error?
7. What is the difference between HOSVD and st-HOSVD?
8. How is addition represented in Tucker format, and why do ranks grow?
9. How are scalar products computed through small Gram matrices and core contractions?
10. Why do orthonormal factor matrices make norms and errors easy?
11. How does QR orthogonalization transfer non-orthogonality into the core?
12. How does Tucker recompression reduce ranks without forming the full tensor?

Reference List Used from the Uploaded Guide

The following sources are the Tucker-related references listed in the uploaded exam guide:

- **DL00** De Lathauwer, L., De Moor, B., Vandewalle, J. *A multilinear singular value decomposition.* SIAM J. Matrix Anal. Appl. 21(4), 1253–1278, 2000.

- **GVL13** Golub, G. H., Van Loan, C. F. *Matrix Computations.* 4th ed. Johns Hopkins University Press, 2013.
- **KB09** Kolda, T. G., Bader, B. W. *Tensor decompositions and applications.* SIAM Review 51(3), 455–500, 2009.
- **T66** Tucker, L. R. *Some mathematical notes on three-mode factor analysis.* Psychometrika 31(3), 279–311, 1966.
- **TB97** Trefethen, L. N., Bau, D. *Numerical Linear Algebra.* SIAM, 1997.
- **VVM12** Vannieuwenhoven, N., Vandebril, R., Meerbergen, K. *A new truncation strategy for the higher-order singular value decomposition.* SIAM J. Sci. Comput. 34(2), A1027–A1052, 2012.